

Assmaa Aziz  
 Param Gandhi  
 STAT 151A  
 19 December 2025

## Final Project: Predicting Spotify Song Popularity Score

### 1. Introduction

We built a regression model that predicts a track's Spotify popularity score to help music producers and labels decide which artists are the best candidates to pitch a new song to. We worked with the *Spotify Songs for ML & Analysis (8700+ tracks)* dataset from Kaggle, uploaded by user AlyAhmedTS13, which captures the metadata about songs, artists, and albums using Spotify's API. Our project compares artist characteristics, such as popularity, follower count, and genre, with track and album characteristics, like duration, explicit content, album type, and release date, to assess how strongly an artist's popularity predicts the expected popularity of a track. Our research question is: How well do artist characteristics explain a song's popularity on Spotify compared to track and album characteristics, and how can these predictors be used to estimate expected song popularity scores for a given track across different artists?

We focused on the following columns from the dataset as predictors:

- *artist\_popularity*: Popularity score of the artist (0–100).
- *artist\_followers*: Number of followers the artist has on Spotify.
- *artist\_genres*: List of genres associated with the artist.
- *track\_number*: Position of the track within its album or release.
- *track\_duration\_min*: Duration of the track in minutes.
- *explicit*: Boolean flag indicating whether the track contains explicit content.
- *album\_type*: Type of album release (e.g., album, single, compilation).
- *album\_total\_tracks*: Total number of tracks in the album.
- *album\_release\_date*: Release date of the album (YYYY-MM-DD). From this, we created a new variable containing the days since release to better capture how recently a track came out.

We split the data into training and test sets and fit four different multiple linear regression models on the training data:

- **Artist**: a model which includes artist-only features, including artist popularity, artist followers, and artist genres.
- **Track**: a model with track-only features, including track number, track duration, explicit, album type, album total tracks, and days since release.
- **Full**: a model that included all of the features found in the artist and track models.
- **Interaction**: a model that included the features from the full model, as well as the interactions between artist popularity and explicit content, album type, and days since release.

We used *track\_popularity* as our response, which is the popularity score assigned by Spotify (0–100), based on recent streaming activity. We chose these predictors because they all provide information that may be useful in predicting how well a song performs on Spotify. Artist popularity, follower counts, and genre affiliations reflect an artist's fanbase. Song duration and

whether a song contains explicit content impacts listening frequency and placement on major playlists. Album type, release date, track number, and total tracks captures how a song is released and positioned, affecting visibility and streaming patterns since songs that appear earlier on an album typically receive more streams. We explored interactions to capture instances where the effect of track features on song popularity is dependent on an artist's popularity. We focused on interactions that we hypothesized would impact track popularity. Explicit content, album type, and recency can affect song popularity differently for lesser known artists than highly popular artists.

For each of these models and using the test set, we computed the root mean square error (RMSE), the squared correlation coefficient ( $R^2$ ), and prediction intervals. We selected the best model primarily based on its measure of accuracy (RMSE), while also considering precision as measured by the average width of the model's prediction intervals. We looked at the test  $R^2$  as a sanity check to see how much variability in track popularity is explained by the model's predictions on unseen data. The analysis explaining the selection process of the final model is included in the Additional Work section, while the remaining sections focus solely on the final model.

## 2. Final Regression Model

Our final regression model is the interaction model, which predicts Spotify track popularity score using all of the predictors mentioned previously, as well as the interactions between artist popularity and album type, explicitness, and days since release. This model aims to provide a nuanced prediction of track popularity that accounts for both main effects and conditional relationships between artist and track characteristics.

$$\begin{aligned} \hat{y}_i = & \hat{\alpha} + \hat{\beta}_1 \text{artist\_popularity}_i + \hat{\beta}_2 \log(\text{artist\_followers})_i + \sum_g^{24} \hat{\beta}_{g+2} \text{genre}_{g,i} \\ & + \hat{\beta}_{27} \text{track\_duration\_min}_i + \hat{\beta}_{28} \text{track\_number}_i + \hat{\beta}_{29} \log(\text{album\_total\_tracks})_i + \hat{\beta}_{30} \text{album\_type\_single}_i \\ & + \hat{\beta}_{31} \text{album\_type\_compilation}_i + \hat{\beta}_{32} \text{explicit}_i + \hat{\beta}_{33} \text{days\_since\_release}_i \\ & + \hat{\gamma}_1 (\text{artist\_popularity}_i \times \text{album\_type\_single}_i) + \hat{\gamma}_2 (\text{artist\_popularity}_i \times \text{album\_type\_compilation}_i) \\ & + \hat{\gamma}_3 (\text{artist\_popularity}_i \times \text{explicit}_i) + \hat{\gamma}_4 (\text{artist\_popularity}_i \times \text{days\_since\_release}_i) \end{aligned}$$

Note that genre contains the 24 top genres since the dataset includes 430 unique genres. A log transformation was applied to artist followers and album total tracks to account for the large differences in scale between very small and very large artists and between short and long albums (i.e. EP's vs compilation albums).

## 3. Discussion

In order to create prediction intervals for the interaction model, we checked if the assumptions of the Normal Linear Model (NLM) are valid. From the Residuals vs Fitted Values plot (Figure 1a), we found that heteroskedasticity occurs since the spread of residuals increases as fitted values increase. To assess the linearity assumption, which requires  $E(e_i|x_i) = 0$ , we checked the average of the residuals in slices. The binned residual analysis (Figure 1b), which is 10 quantile-based bins, shows that the mean residuals range from approximately -1.4 to 2.8, which is minimal relative to the popularity scale (0–100). This suggests that the relationship is approximately linear, making our model reasonable for prediction purposes. From the Q-Q plot

(Figure 1c), we see an S-shaped curve, implying that the residuals are not normally distributed. The left tail is significantly below the line, implying a left skew. These observations tell us that the homoscedasticity and normality assumptions of the NLM are violated. These assumptions are not necessary for a model to perform well at prediction, however it makes our prediction intervals unreliable. To overcome this, we created prediction intervals using bootstrap cases.

Using the interaction model to predict the fitted values for the test set yielded an RMSE of 18.75 with an  $R^2$  of 0.26. This indicates that on average, predicted track popularity scores differed from the observed values by about 19 points. With scores ranging between 0-100 and track popularity in the test set having a standard deviation of 21.76, an error of 19 points is reasonable.  $R^2$  tells us that the combination of the artist, track, and interaction features explain about 26% of the variance in track popularity. Our average 95% bootstrap prediction interval for the test set was [3.59, 82.47] with a mean interval width of 78.88, suggesting large uncertainty.

While holding track features fixed, we examined how the interaction model could be used to predict track popularity across different artists for a hypothetical new song. To illustrate this, we created a representative fixed track by setting numeric features to their median value and binary/categorical features to their reference group values. We selected artists with low, medium, and high artist popularity from the training data, who are Lafawndah, Tim McGraw, and Taylor Swift respectively. Based on the predictions made by this model (Table 1), Taylor Swift has the highest track popularity for this fixed song, with a predicted popularity score of 62.21. Across artists, predicted track popularity drastically increases with artist popularity.

While the interaction model predicted track popularity scores best compared to the other models we explored, it has several limitations. It has moderate predictive power, since its RMSE is only slightly lower than the standard deviation of track popularity on a 0-100 scale. Its low  $R^2$  value indicates that 74% of the variability in track popularity for unseen data remains unexplained by the model's predictions. An average prediction interval width of about 79 points indicates substantial uncertainty and a lack of precision in our predictions. These limitations are unsurprising, considering the inherent uncertainty in predicting song popularity and the absence of critical factors such as playlist placement, marketing, virality, and cultural trends, which cannot be captured by the Spotify metadata.

#### 4. Conclusion

Overall, our analysis found that artist features explain substantially more variation in track popularity on Spotify than track features, and the model incorporating both main effects and the interactions yields the best performance. Of the four models we examined (artist-only, track-only, full, and interaction), the interaction model has the lowest test RMSE, the highest test  $R^2$ , and the most narrow prediction intervals, indicating it's the model with the most accuracy and precision. When treating track features as fixed, we found that across artists, predicted track popularity increases as predicted artist popularity increases. This model can be used with a fixed track to compare expected popularity scores for the same song across artists, making it helpful for producers in deciding between artists. However, due to its moderate predictive power, considerable uncertainty, and lack of precision, as illustrated by its wide prediction intervals and low test  $R^2$ , this model should be used cautiously. It can be used to support preliminary screening but it should be combined with market considerations when deciding which artists are the best candidates to pitch a new song to.

## 5. Additional Work

As stated in the introduction, we fit four models in the following order: the artist-only, track-only, full, and the interaction model that we ultimately chose. This section describes the other models we tried and the diagnostics we used to determine our final model.

### Artist Model

#### 1. Purpose & Process

We first created an artist-only model to assess how artist-specific features influence track popularity scores on Spotify. The predictor variables include the top 24 genres associated with artists, artist popularity, and log-transformed follower counts. The top genres were determined by the genres that appear at least 100 times in the dataset, since the original data contains 430 unique genres and that many dummy terms would make the model too complex, risking overfitting. To simplify the complexity added to the model due to the 24 genre dummy variables and to see whether a simpler model makes better predictions, we used LASSO regularization to perform feature selection. We used 10-fold cross-validation on the training set to identify the minimum regularization coefficient. Using that lambda value, we fit a LASSO model to set the weights of less important features to 0. Then we extracted the names of the predictors that LASSO kept. Using the vector of names, we fit an ordinary least squares model containing the predictors selected by LASSO. However, we found that none of the predictor coefficients were set to zero. As a result, our ordinary least squares model contained all 24 of the genre variables, the artist popularity term, and the log transformed follower counts term. This model served as the artist-only baseline for predicting track popularity without incorporating track/album-specific characteristics.

#### 2. Checking Assumptions

All of the diagnostic plots, except for the binned residual analysis, look similar for each model we tried. Identical conclusions can be made for the reasons explained earlier when we discussed the interaction model.

The Residuals vs Fitted Values plot (Figure 2a) for the artist model suggests heteroskedasticity. The binned residual analysis (Figure 2b) shows mean residuals ranging from approximately -2.8 to 2.5, suggesting that the relationship is roughly linear and making our model reasonable for prediction. The Q-Q plot (Figure 2c) implies that the residuals are not normally distributed and that they're skewed left.

This indicates that the assumptions of the normal linear model are violated, which would make inaccurate prediction intervals. These assumptions are not necessary for prediction, so evaluating our model's predictive error is still valid. However, in order to use prediction intervals, we must rely on the normal linear model. As a result, to construct prediction intervals for this data, we'll use bootstrap cases.

#### 3. Performance

Using the artist-only model to predict the fitted values for the test set yielded an RMSE of 19.51 with an  $R^2$  of 0.196, indicating that artist features alone explain about 20% of the

variance in track popularity. Our error is reasonable since the standard deviation of track popularity in the test set is 21.76. Our average 95% bootstrap prediction interval for the test set was [2.94, 82.46] with a mean interval width of 79.52. This suggests high uncertainty and low precision for the predictions by the artist model.

## Track Model

### 1. Purpose & Process

Next, we created a track model to determine how track- and album-level features explain the variation in track popularity scores. The predictor variables include track number (the position of the track on the album), track duration in minutes, whether or not the track contains explicit content, album type (single, album, or compilation), days since release, and log transformed total number of tracks in the album. Similar to the artist model, to find a simpler model that could possibly better predict track popularity, we used LASSO regularization to perform feature selection. We used 10-fold cross-validation on the training set to calculate the regularization coefficient, fit LASSO to see which predictor coefficients were set to 0, and extracted the names of the features which LASSO did not remove. Using those features selected by LASSO, we fit an ordinary least squares model. However, once again, LASSO didn't remove any predictors, so the track model contained all of the original predictors. This model served as the baseline for predicting track popularity using only track and album characteristics.

### 2. Checking Assumptions

All of the diagnostic plots, except for the binned residual analysis, look similar for each model we tried. Identical conclusions can be made for the reasons explained earlier when we discussed the interaction model.

The Residual Vs Fitted Values plot (Figure 3a) suggests heteroskedasticity of the residuals for the track-only model. The binned residual analysis (Figure 3b) shows mean residuals ranging from approximately -3.3 to 3.0, suggesting that the relationship reflected by the model is approximately linear. The Q-Q plot (Figure 3c) indicates that the distribution of the residuals are not normal and skewed left. The normality and homoskedastic assumptions of the normal linear model are violated, but the linearity remains sound, making our model fine for prediction. Reliable prediction intervals rely on these assumptions, which is why we constructed prediction intervals using the bootstrap cases.

### 3. Performance

Using the track/album model to predict the fitted values for the test set yielded an RMSE of 20.42 with an  $R^2$  of 0.118, indicating that track/album-level features alone explain about 12% of the variance in track popularity. Once again, the error is reasonable since it's similar to the test standard deviation of track popularity. Our average 95% bootstrap prediction interval for the test set was [0.68, 84.89] with a mean interval width of 84.21. Wider prediction intervals suggest more uncertainty and less precision in the predictions made by the track model when compared to the artist model.

## Full Model

### 1. Purpose & Process

The full model was constructed to assess the combined effect of both artist-level and track-level predictors on track popularity. Unlike the previous models, all predictors selected in the artist-only and track-only models were included, so there was no need to apply any LASSO regularization. We did this to truly evaluate the effect of combining the two sets of features and wanted to consider the influence of every single predictor, rather than use the subset of LASSO-produced predictors. This model provides a baseline for understanding whether combining artist and track information improves predictive performance.

### 2. Checking Assumptions

The patterns present in the Residuals vs Fitted Values (Figure 4a) and Q-Q plots (Figure 4c) for the full model are extremely similar to the models previously mentioned. In summary, we see heteroskedastic, non-normal residuals that suggest a left skew. To examine linearity, the binned residual analysis (Figure 4b) shows mean residuals ranging from approximately -1.2 to 1.0, suggesting that the predictors and our response approximately have a linear relationship. These diagnostics tell us the normal linear model is violated. Since linearity is preserved, our model is reasonable to use for prediction. Because prediction intervals rely on the assumptions of the normal linear model, we've computed them through bootstrap cases.

### 3. Performance

Using the full model to predict the fitted values for the test set yielded an RMSE of 19.51 with an  $R^2$  of 0.241, indicating that combination of the artist-level and track/album-level features combined (without interactions) explain about 24% of the variance in track popularity. Our average 95% bootstrap prediction interval for the test set was [2.97, 82.47] with a mean interval width of 79.50.

In general the model performance improves as more structure and context is added (Table 2). The track/album-only model performs the worst when fit to the test set, with the largest RMSE and average prediction interval width, while the artist-only model performs substantially better, indicating that artist characteristics are much stronger predictors of popularity than track features alone. Adding track-level variables to the artist model yields minimal gains in terms of test set RMSE and prediction interval width, suggesting limited additive value when effects are assumed to be uniform across artists. The interaction model performs best in the test set, having the lowest RMSE and narrowest prediction intervals as well as the largest  $R^2$ . This indicates that track features matter primarily through their interaction with artist popularity. We decided on our final predictive model mostly based on model accuracy (RMSE), with average prediction interval width and  $R^2$  being additional, supportive measures to check for uncertainty and the amount of variability in track popularity explained by the model. We observed that as error decreased, the uncertainty decreased, and explained variance increased. These support our observations that as the model gained more structure, accuracy and precision improved. We chose the interaction model as the best predictor of track popularity

since it best captures the context-dependence and provides the best balance of predictive accuracy, precision, and interpretability.

## 6. References

AlyAhmedTS13. (2025). *Spotify Songs for ML & Analysis (8700+ tracks)*. Kaggle.

<https://www.kaggle.com/datasets/alyahmedts13/spotify-songs-for-ml-and-analysis-over-8700-tracks>



## Appendix

### Tables

**Table 1.** The predicted track popularity scores when a fixed track is pitched to artists of varying popularity levels (low, mid size, high).

| artist_name<br><chr> | predicted_track_pop<br><dbl> | artist_popularity<br><dbl> | log_artist_followers<br><dbl> |
|----------------------|------------------------------|----------------------------|-------------------------------|
| Lafawndah            | 13.73125                     | 16                         | 9.96801                       |
| Tim McGraw           | 55.93946                     | 69                         | 15.43342                      |
| Taylor Swift         | 62.20893                     | 100                        | 18.79598                      |

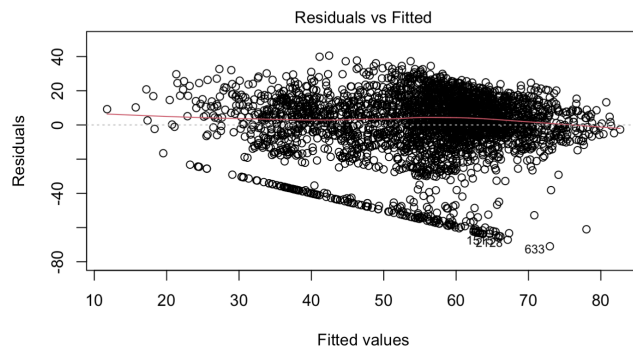
**Table 2.** A summary of our model comparison to determine the best predictive model.

| models<br><chr> | RMSE<br><dbl> | R2<br><dbl> | avg_PI_width<br><dbl> |
|-----------------|---------------|-------------|-----------------------|
| interaction     | 18.75385      | 0.2567235   | 78.87647              |
| full            | 18.94407      | 0.2414337   | 79.50054              |
| artist-only     | 19.51403      | 0.1960646   | 79.51638              |
| track-only      | 20.42139      | 0.1192480   | 84.21112              |

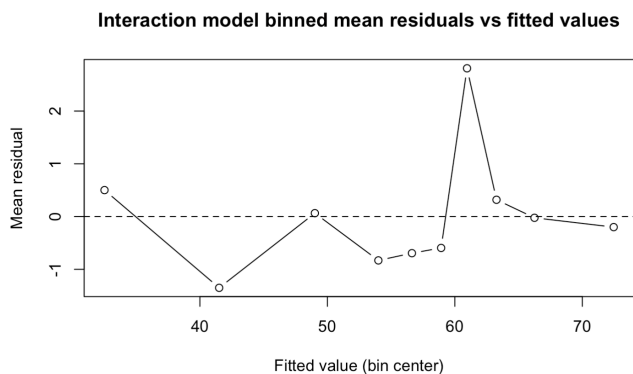
### Figures

#### Interaction

**Figure 1a.** The Residuals vs Fitted plot for the interaction model.

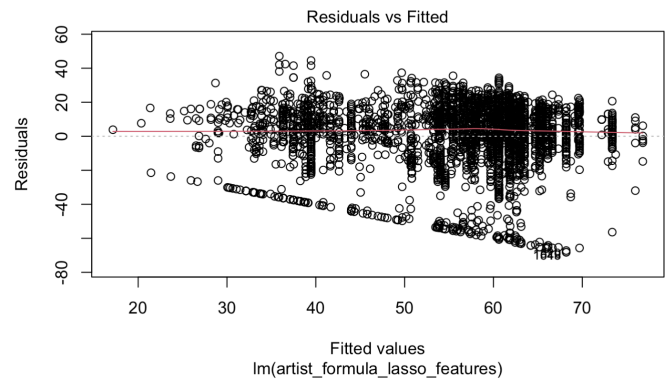


**Figure 1b.** The Binned Mean Residuals vs Fitted plot for the interaction model.

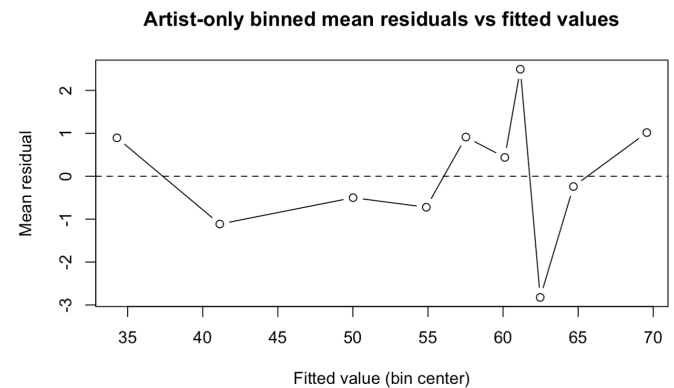


#### Artist

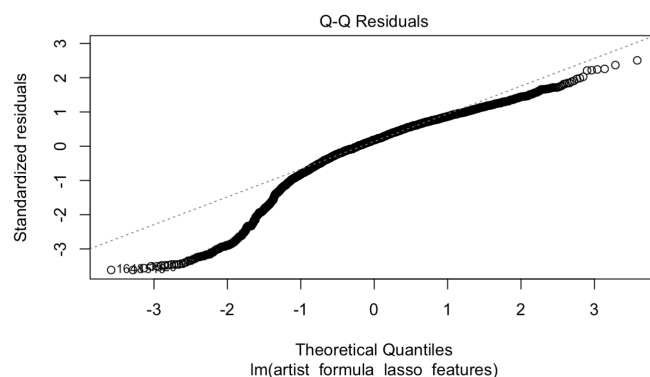
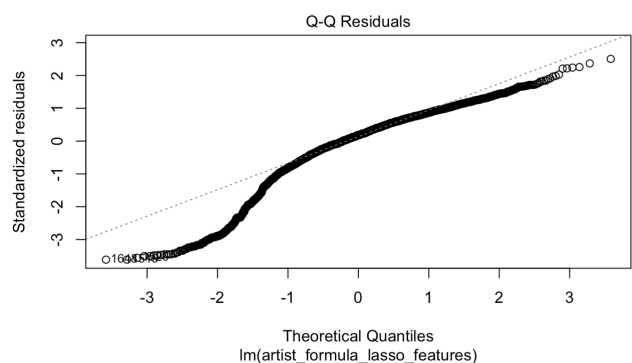
**Figure 2a.** The Residuals vs Fitted plot for the artist model.



**Figure 2b.** The Binned Mean Residuals vs Fitted plot for the artist model.

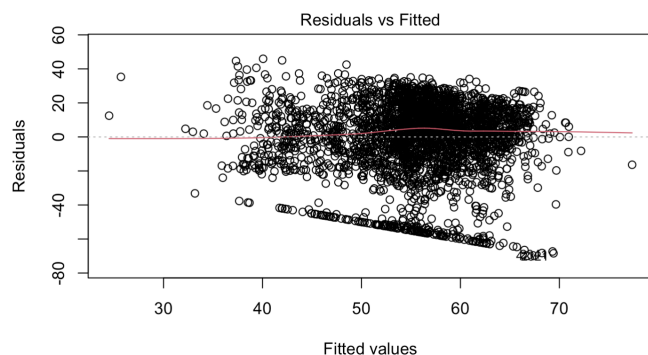


**Figure 1c.** The Q-Q plot for the interaction model. **Figure 2c.** The Q-Q plot for the artist model.

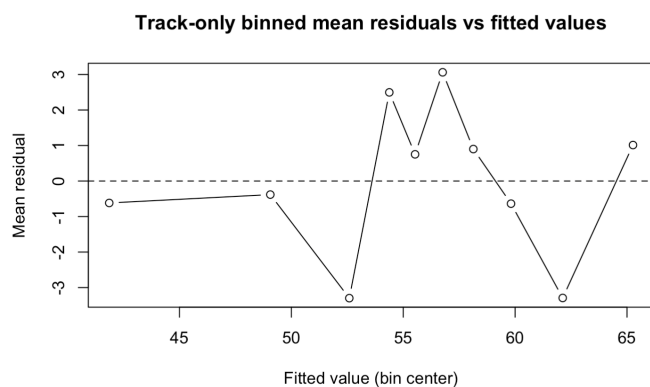


### Track

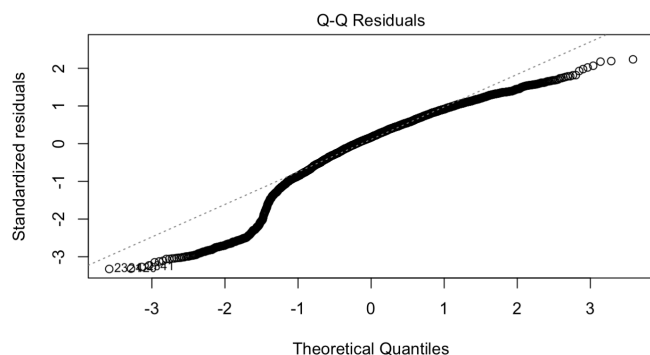
**Figure 3a.** The Residuals vs Fitted plot for the track model.



**Figure 3b.** The Binned Mean Residuals vs Fitted plot for the track model.

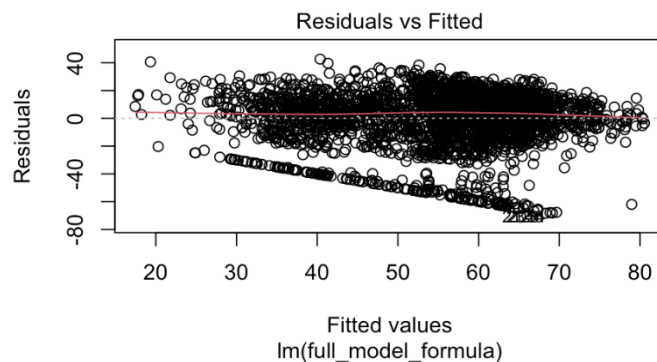


**Figure 3c.** The Q-Q plot for the track model.

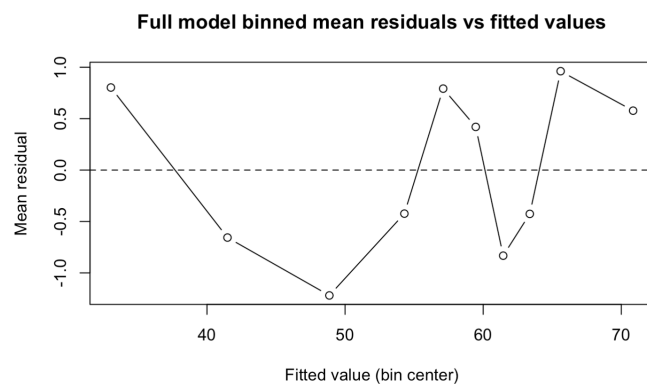


### Full

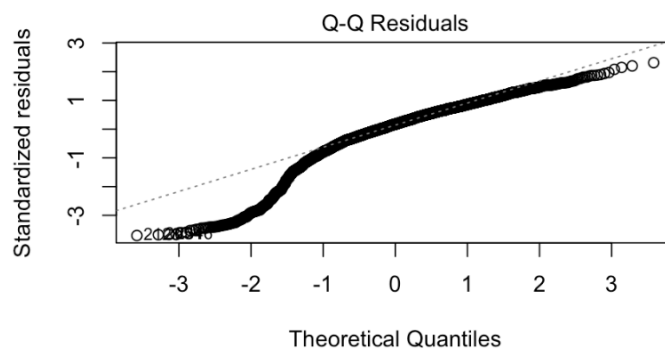
**Figure 4a.** The Residuals vs Fitted plot for the full model.



**Figure 4b.** The Binned Mean Residuals vs Fitted plot for the full model.



**Figure 4c.** The Q-Q plot for the full model.



# Code Appendix

## Top occurring genres

```
#load data
library(tidyverse)
library(glmnet)
library(tidyr)

set.seed(151)
spotify_data = read_csv('spotify_data_clean.csv')

#Genres that occur in the dataset at least 100 times
#separate list of genres into rows, remove whitespace at beginning or
#end of genre name, remove NA, find counts
genre_counts = spotify_data %>%
  separate_rows(artist_genres, sep = ',') %>%
  mutate(artist_genres = str_trim(artist_genres)) %>%
  filter(artist_genres != 'N/A') %>%
  group_by(artist_genres) %>%
  count() %>%
  arrange(desc(n))

genre_counts
```

```
# A tibble: 430 x 2
# Groups:   artist_genres [430]
  artist_genres      n
  <chr>           <int>
1 pop             920
2 country         680
3 soundtrack      450
```

```

4 hip hop      431
5 indie        376
6 folk         337
7 rock         327
8 rap          297
9 alternative pop 228
10 soft pop    213
# i 420 more rows

```

```
cat('Number of unique genres: \n', length(genre_counts$artist_genres))
```

```

Number of unique genres:
430

```

```

top_genres = genre_counts %>% filter(n >= 100)
top_genres

```

```

# A tibble: 24 x 2
# Groups:   artist_genres [24]
  artist_genres      n
  <chr>           <int>
1 pop             920
2 country         680
3 soundtrack      450
4 hip hop         431
5 indie           376
6 folk            337
7 rock            327
8 rap             297
9 alternative pop  228
10 soft pop       213
# i 14 more rows

```

```
top_genre_names = top_genres$artist_genres
```

**Filter, one hot encode, apply transformations to the original data and train/test split**

```

#data cleaning
#filter original data to only include the top genres to create one hot
#encoding (OHE) columns for the genres
spotify_genres_filtered = spotify_data %>% separate_rows(artist_genres,
                                                         sep = ',') %>%

  mutate(artist_genres = str_trim(artist_genres)) %>%
  filter (artist_genres %in% top_genre_names)

#OHE: give value of 1 if song is from artist associated with that genre
#and 0 if otherwise
spotify_OHE_cols = spotify_genres_filtered %>% mutate(value = 1) %>%
  pivot_wider(id_cols = track_id, names_from = artist_genres,
              values_from = value,
              values_fill = 0)

#make genre column names without spaces so we can use it in our
#regression equation later
new_col_names = make.names(names(spotify_OHE_cols))
names(spotify_OHE_cols) = new_col_names

#add OHE columns to original data set to keep one track per row
spotify_filtered = spotify_OHE_cols %>%
  inner_join(spotify_data %>% select(-artist_genres), by = 'track_id')

#apply log transformation to artist_followers
spotify_filtered$log_artist_followers = log(spotify_filtered$artist_followers)

#treat explicit and album type as factors
spotify_filtered$explicit = factor(spotify_filtered$explicit)
spotify_filtered$album_type = factor(spotify_filtered$album_type)

#treat album_release_date as days since release
spotify_filtered$days_since_release = as.numeric(Sys.Date() -
                                                  spotify_filtered$album_release_date)

#our final filtered and transformed data we're working with
spotify_filtered

```

```

# A tibble: 3,683 x 40
  track_id      dark.r.b pop.punk indie  edm country  pop  folk egyptian.pop
  <chr>          <dbl>    <dbl> <dbl> <dbl>   <dbl> <dbl> <dbl>         <dbl>

```

```

1 15xptTfRBrjsp~      1      0      0      0      0      0      0      0
2 4ccpCcZYseq8V~      1      0      0      0      0      0      0      0
3 1YEZbdT417Sfo~      1      0      0      0      0      0      0      0
4 2zzjpwG8wu3sk~      0      1      0      0      0      0      0      0
5 1NXbNEAcPvY5G~      0      0      1      0      0      0      0      0
6 0vXBmU9Zus9Zt~      1      0      0      0      0      0      0      0
7 3Dy4xsff7c0fR~      0      0      0      1      0      0      0      0
8 31TXxq8gfgYyr~      0      0      1      0      1      1      1      0
9 5eXgqtg3T8Av0~      0      0      1      0      1      1      1      0
10 62V2ZHslgQV98~      0      0      1      0      1      1      1      0
# i 3,673 more rows
# i 31 more variables: soft.pop <dbl>, pop.rock <dbl>, alternative.pop <dbl>,
#   art.pop <dbl>, alternative.r.b <dbl>, r.b <dbl>, rap <dbl>,
#   soundtrack <dbl>, rock <dbl>, hip.hop <dbl>, nu.metal <dbl>, r.b.pop <dbl>,
#   synthpop <dbl>, indie.pop <dbl>, grunge <dbl>, classic.rock <dbl>,
#   track_name <chr>, track_number <dbl>, track_popularity <dbl>,
#   explicit <fct>, artist_name <chr>, artist_popularity <dbl>, ...

```

```

#train/test split
sample = sample(nrow(spotify_filtered), 0.2 * nrow(spotify_filtered))
train = spotify_filtered[-sample,]
test = spotify_filtered[sample,]

```

## Artist-only model

### Artist-only: LASSO for feature selection

```

#k folds cross validation to find lambda value for LASSO
#design matrix and response vector
top_genre_names_new = new_col_names[-1]
artist_predictors = c(top_genre_names_new, 'artist_popularity',
                      'log_artist_followers')
predictors_formula = paste(artist_predictors, collapse = ' + ')
artist_formula = as.formula(paste('track_popularity ~ ', predictors_formula))

#create design matrix and response vector
x_train = model.matrix(artist_formula, data = train)[, -1]
y_train = train$track_popularity

#cross validation to find lambda value

```

```

cv_lasso = cv.glmnet(x_train, y_train, alpha = 1, nfolds = 10, standardize = TRUE)

#LASSO model using lambda value found
artist_lasso = glmnet(x_train, y_train, alpha = 1, lambda = cv_lasso$lambda.min,
                      standardize = TRUE)

cat('LASSO artist slope coefficients: \n')

```

LASSO artist slope coefficients:

```

lasso_coefs = coef(artist_lasso)
lasso_coefs

```

27 x 1 sparse Matrix of class "dgCMatrix"

|                   | s0          |
|-------------------|-------------|
| (Intercept)       | 38.8678107  |
| dark.r.b          | -11.2418708 |
| pop.punk          | -4.1879303  |
| indie             | 6.4022494   |
| edm               | -4.5338668  |
| country           | -3.1688014  |
| pop               | -3.1482243  |
| folk              | -15.2920403 |
| egyptian.pop      | -11.7540296 |
| soft.pop          | -9.7675297  |
| pop.rock          | -4.2475097  |
| alternative.pop   | 5.0564017   |
| art.pop           | -10.2490610 |
| alternative.r.b   | 0.3877979   |
| r.b               | -3.2211789  |
| rap               | -4.3876666  |
| soundtrack        | -21.1766708 |
| rock              | -4.2152492  |
| hip.hop           | -2.5980303  |
| nu.metal          | -6.0527800  |
| r.b.pop           | -9.5243013  |
| synthpop          | 2.8210721   |
| indie.pop         | -9.9951821  |
| grunge            | -20.9063085 |
| classic.rock      | -2.5052370  |
| artist_popularity | 0.8229003   |

```
log_artist_followers -2.4109679
```

```
#extract LASSO slope coefficients and find the predictors where
#the coefficient != 0
lasso_coefs_values = lasso_coefs[-1, ]
lasso_predictors = names(lasso_coefs_values[lasso_coefs_values != 0])
cat('\n Artist features selected by LASSO: \n')
```

Artist features selected by LASSO:

```
lasso_predictors
```

```
[1] "dark.r.b"          "pop.punk"          "indie"
[4] "edm"              "country"           "pop"
[7] "folk"             "egyptian.pop"      "soft.pop"
[10] "pop.rock"         "alternative.pop"    "art.pop"
[13] "alternative.r.b"   "r.b"               "rap"
[16] "soundtrack"       "rock"              "hip.hop"
[19] "nu.metal"         "r.b.pop"           "synthpop"
[22] "indie.pop"        "grunge"            "classic.rock"
[25] "artist_popularity" "log_artist_followers"
```

```
#variable selection using LASSO, artist-only model
```

### Artist-only: fitting OLS model with features chosen by LASSO

```
#regression equation for features selected by LASSO
lasso_predictors_formula = paste(lasso_predictors, collapse = ' + ')
artist_formula_lasso_features = as.formula(paste('track_popularity ~ ',
                                                  lasso_predictors_formula))

#artist-only model
artist = lm(artist_formula_lasso_features, data = train)
summary(artist)
```

Call:



```
lm(formula = artist_formula_lasso_features, data = train)
```

Residuals:

| Min     | 1Q     | Median | 3Q     | Max    |
|---------|--------|--------|--------|--------|
| -68.190 | -7.687 | 3.382  | 12.859 | 47.109 |

Coefficients:

|                      | Estimate  | Std. Error | t value | Pr(> t ) |     |
|----------------------|-----------|------------|---------|----------|-----|
| (Intercept)          | 39.21805  | 5.06307    | 7.746   | 1.30e-14 | *** |
| dark.r.b             | -11.52938 | 2.68288    | -4.297  | 1.78e-05 | *** |
| pop.punk             | -4.20860  | 3.49846    | -1.203  | 0.22908  |     |
| indie                | 6.41322   | 2.73253    | 2.347   | 0.01899  | *   |
| edm                  | -4.73186  | 2.43803    | -1.941  | 0.05237  | .   |
| country              | -3.25682  | 1.50279    | -2.167  | 0.03030  | *   |
| pop                  | -3.31276  | 1.57497    | -2.103  | 0.03552  | *   |
| folk                 | -15.39976 | 3.07857    | -5.002  | 6.00e-07 | *** |
| egyptian.pop         | -11.70644 | 2.57979    | -4.538  | 5.92e-06 | *** |
| soft.pop             | -9.88827  | 2.03095    | -4.869  | 1.18e-06 | *** |
| pop.rock             | -4.52164  | 4.24358    | -1.066  | 0.28673  |     |
| alternative.pop      | 5.12202   | 2.87727    | 1.780   | 0.07515  | .   |
| art.pop              | -10.40654 | 1.91281    | -5.440  | 5.75e-08 | *** |
| alternative.r.b      | 0.79053   | 4.17979    | 0.189   | 0.85000  |     |
| r.b                  | -3.36920  | 2.14391    | -1.572  | 0.11617  |     |
| rap                  | -4.63620  | 2.03408    | -2.279  | 0.02272  | *   |
| soundtrack           | -21.39443 | 1.87361    | -11.419 | < 2e-16  | *** |
| rock                 | -4.32720  | 1.85181    | -2.337  | 0.01952  | *   |
| hip.hop              | -2.56320  | 1.49089    | -1.719  | 0.08568  | .   |
| nu.metal             | -6.19618  | 2.30467    | -2.689  | 0.00722  | **  |
| r.b.pop              | -10.51311 | 6.69331    | -1.571  | 0.11636  |     |
| synthpop             | 3.12220   | 4.65782    | 0.670   | 0.50271  |     |
| indie.pop            | -10.28621 | 3.13104    | -3.285  | 0.00103  | **  |
| grunge               | -20.96342 | 2.66953    | -7.853  | 5.66e-15 | *** |
| classic.rock         | -2.58119  | 2.41076    | -1.071  | 0.28439  |     |
| artist_popularity    | 0.84396   | 0.09778    | 8.631   | < 2e-16  | *** |
| log_artist_followers | -2.52394  | 0.61124    | -4.129  | 3.74e-05 | *** |

---

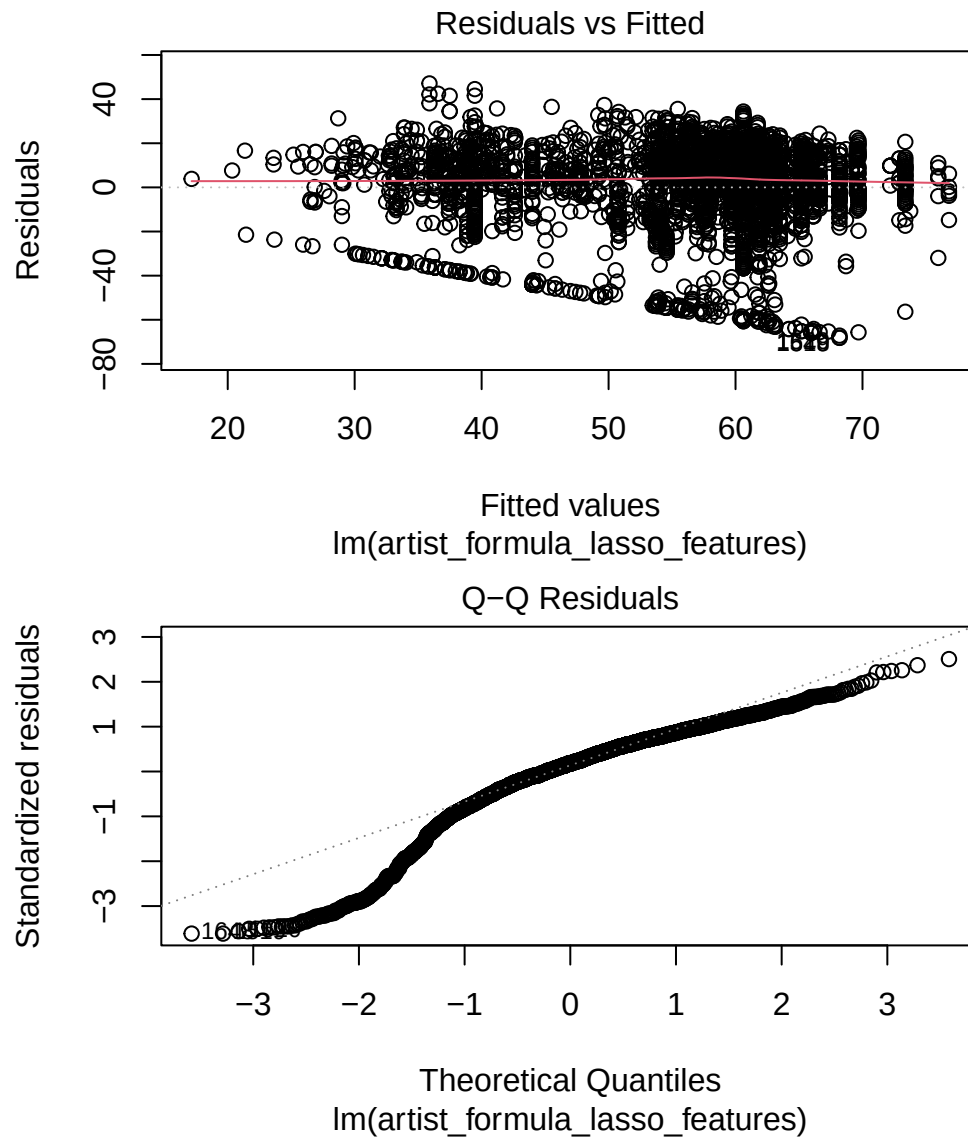
Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

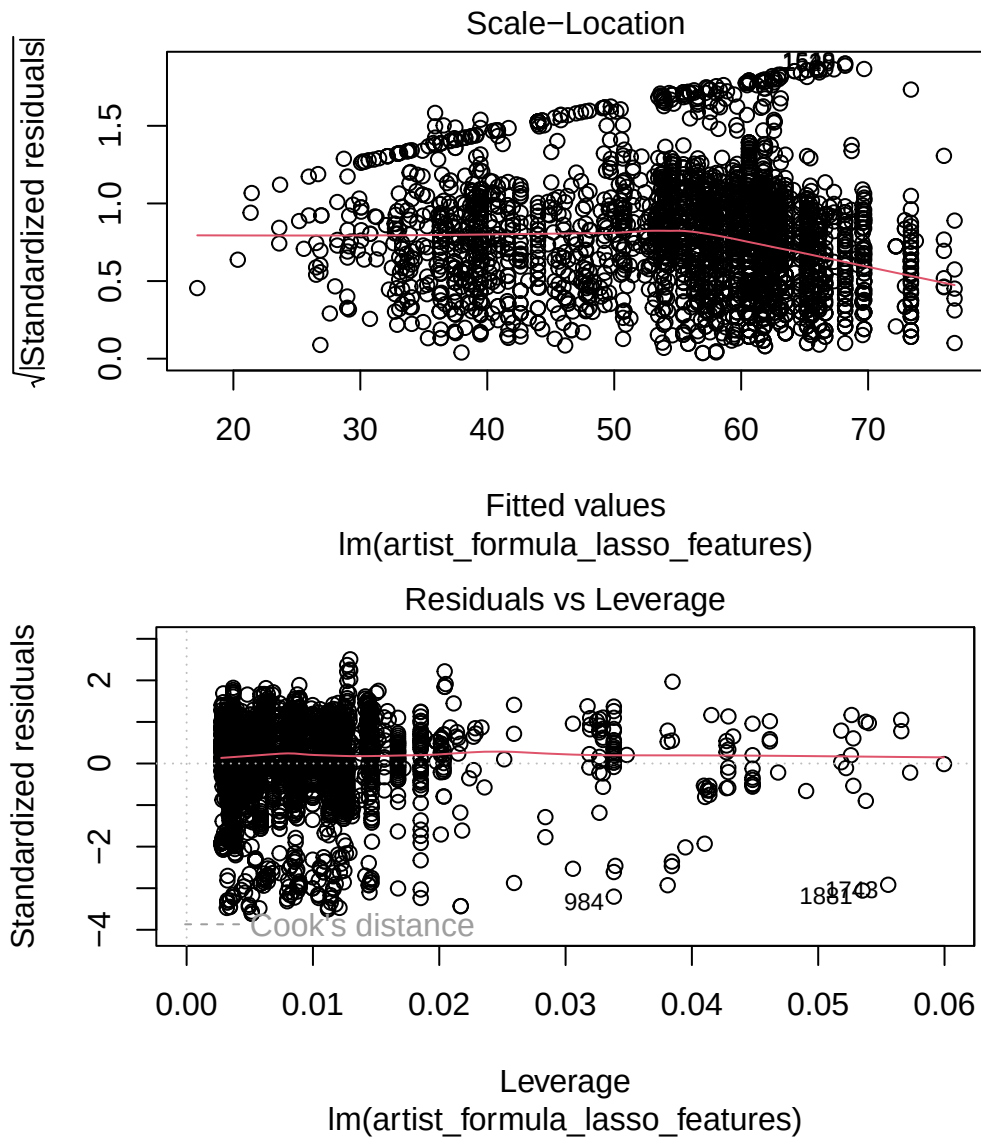
Residual standard error: 18.92 on 2920 degrees of freedom

Multiple R-squared: 0.2383, Adjusted R-squared: 0.2315

F-statistic: 35.13 on 26 and 2920 DF, p-value: < 2.2e-16

```
plot(artist)
```





### Artist-only: checking assumptions of NLM

Assumptions violated.

```
resid <- residuals(artist)
fitted <- fitted(artist)

n_bins <- 10
```

```

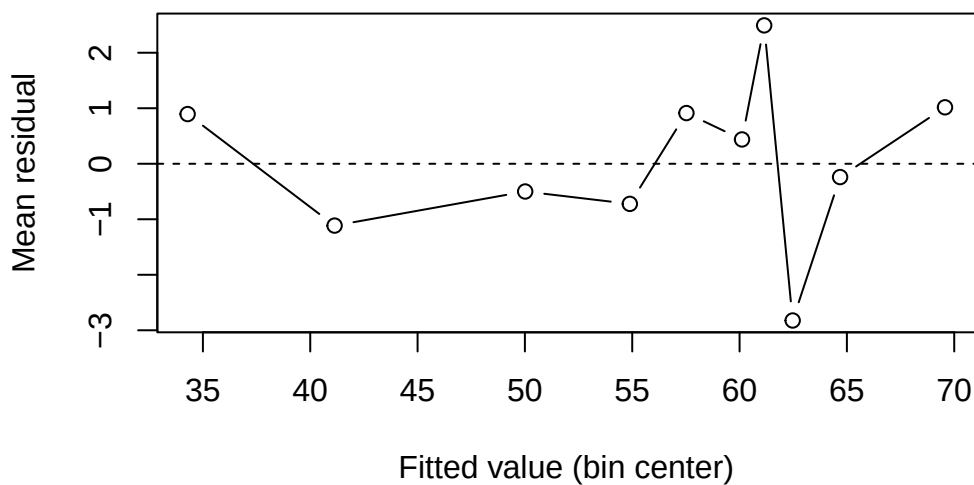
bins <- cut(fitted, breaks = quantile(fitted, probs =
                                     seq(0,1,length.out=n_bins+1)),
           include.lowest = TRUE)

mean_resid <- tapply(resid, bins, mean)

bin_centers <- tapply(fitted, bins, mean)
plot(bin_centers, mean_resid, type = "b",
     xlab = "Fitted value (bin center)",
     ylab = "Mean residual",
     main = "Artist-only binned mean residuals vs fitted values")
abline(h = 0, lty = 2)

```

### Artist-only binned mean residuals vs fitted values



### Artist-only: predictions

```

#assess predictions of artist-only model
predictions = predict(artist, newdata = test)
artist_rmse = sqrt(mean((test$track_popularity - predictions)^2))
artist_r2 = cor(test$track_popularity, predictions) ^ 2
SD_y_test = sd(test$track_popularity)
cat('SD of track popularity: ', SD_y_test)

```

SD of track popularity: 21.75762

```
cat('\n artist-only RMSE: ', artist_rmse)
```

artist-only RMSE: 19.51403

```
cat('\n artist-only R2: ', artist_r2)
```

artist-only R2: 0.1960646

### Artist-only: prediction intervals

```
#number of replications, 1000 replications for each observation in the test set
rep = 1000
boot_outcomes = matrix(NA, nrow = rep, ncol = nrow(test))

#find y = y_hat + residuals 1000 times
for(i in 1:rep){
  #resample the rows from training data with replacement and find new fit
  boot_sample = sample(nrow(train), size = nrow(train), replace = TRUE)
  boot_train = train[boot_sample, ]
  artist_boot = lm(artist_formula_lasso_features, data = boot_train)
  #find the predictions for the test set using this new fit
  y_hats = predict(artist_boot, newdata = test)
  #to find the new outcomes, randomly sample with replacement the boot residuals
  noise = sample(residuals(artist_boot), size = nrow(test), replace = TRUE)
  #outcome = fitted_values + noise
  boot_outcomes[i, ] = y_hats + noise
}

lower = apply(boot_outcomes, 2, quantile, probs = 0.025)
upper = apply(boot_outcomes, 2, quantile, probs = 0.975)
width = upper - lower
PI = data.frame(lower = lower, upper = upper, width = width)
head(PI)
```

|   | lower     | upper    | width    |
|---|-----------|----------|----------|
| 1 | 11.801458 | 88.85740 | 77.05594 |

```

2 -12.960174 67.01311 79.97328
3 -2.909209 79.59371 82.50292
4 10.332466 88.41742 78.08495
5 11.697094 89.72094 78.02385
6 6.014373 81.38896 75.37458

```

```

artist_avg_width = mean(width)
cat('PI average width for the artist-only model: ',
    artist_avg_width, '\n Artist-only PI average bounds bounds:
    [' , mean(lower), ', ', mean(upper), ']' )

```

```

PI average width for the artist-only model: 79.51638
Artist-only PI average bounds bounds:
[ 2.94431 , 82.46069 ]

```

## Track-only model

### Track-only: fitting LASSO for feature selection

```

#LASSO track-only model

x_train = model.matrix(track_popularity ~ track_number + track_duration_min +
                        explicit + album_type + log(album_total_tracks) +
                        days_since_release, data = train)[, -1]
y_train = train$track_popularity

#cross validation to find lambda value
cv_lasso = cv.glmnet(x_train, y_train, alpha = 1, nfolds = 10, standardize = TRUE)

#LASSO model using lambda value found
track_lasso = glmnet(x_train, y_train, alpha = 1,
                     lambda = cv_lasso$lambda.min, standardize = TRUE)

cat('LASSO track slope coefficients: \n')

```

```

LASSO track slope coefficients:

```

```
lasso_coefs = coef(track_lasso)
lasso_coefs
```

8 x 1 sparse Matrix of class "dgCMatrix"

```
              s0
(Intercept)    7.537163e+01
track_number   -2.061677e-01
track_duration_min 1.121331e+00
explicitTRUE    6.876474e+00
album_typecompilation -9.321339e+00
album_typesingle -2.274412e+01
log(album_total_tracks) -7.222165e+00
days_since_release -5.298463e-04
```

```
#extract LASSO slope coefficients and find the predictors where
#the coefficient != 0
lasso_coefs_values = lasso_coefs[-1, ]
lasso_predictors = names(lasso_coefs_values[lasso_coefs_values != 0])
cat('\n Track features selected by LASSO: \n')
```

Track features selected by LASSO:

```
lasso_predictors
```

```
[1] "track_number"          "track_duration_min"
[3] "explicitTRUE"          "album_typecompilation"
[5] "album_typesingle"      "log(album_total_tracks)"
[7] "days_since_release"
```

### Track-only: fitting model with features chosen by LASSO

```
#track-only model
track = lm(track_popularity ~ track_number + track_duration_min + explicit
           + album_type +
           log(album_total_tracks) +
           days_since_release, data = train)
summary(track)
```

```
Call:
lm(formula = track_popularity ~ track_number + track_duration_min +
    explicit + album_type + log(album_total_tracks) + days_since_release,
    data = train)
```

Residuals:

| Min     | 1Q     | Median | 3Q     | Max    |
|---------|--------|--------|--------|--------|
| -68.410 | -9.651 | 3.499  | 14.311 | 45.938 |

Coefficients:

|                         | Estimate   | Std. Error | t value | Pr(> t )     |
|-------------------------|------------|------------|---------|--------------|
| (Intercept)             | 7.561e+01  | 2.797e+00  | 27.035  | < 2e-16 ***  |
| track_number            | -2.067e-01 | 6.828e-02  | -3.026  | 0.00250 **   |
| track_duration_min      | 1.126e+00  | 3.429e-01  | 3.282   | 0.00104 **   |
| explicitTRUE            | 6.885e+00  | 8.940e-01  | 7.702   | 1.82e-14 *** |
| album_typecompilation   | -9.346e+00 | 1.642e+00  | -5.692  | 1.38e-08 *** |
| album_typesingle        | -2.299e+01 | 2.316e+00  | -9.925  | < 2e-16 ***  |
| log(album_total_tracks) | -7.301e+00 | 8.545e-01  | -8.545  | < 2e-16 ***  |
| days_since_release      | -5.331e-04 | 1.017e-04  | -5.244  | 1.69e-07 *** |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

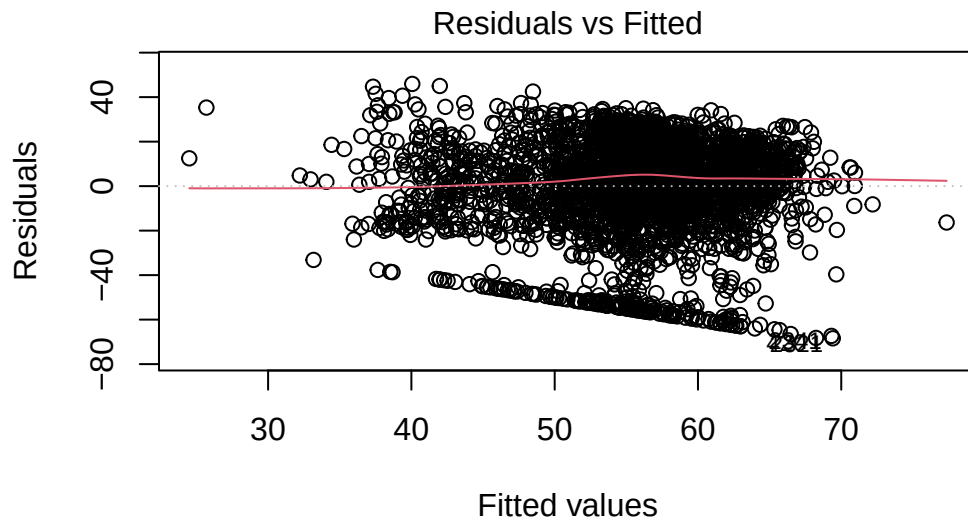
Residual standard error: 20.6 on 2939 degrees of freedom

Multiple R-squared: 0.09047, Adjusted R-squared: 0.0883

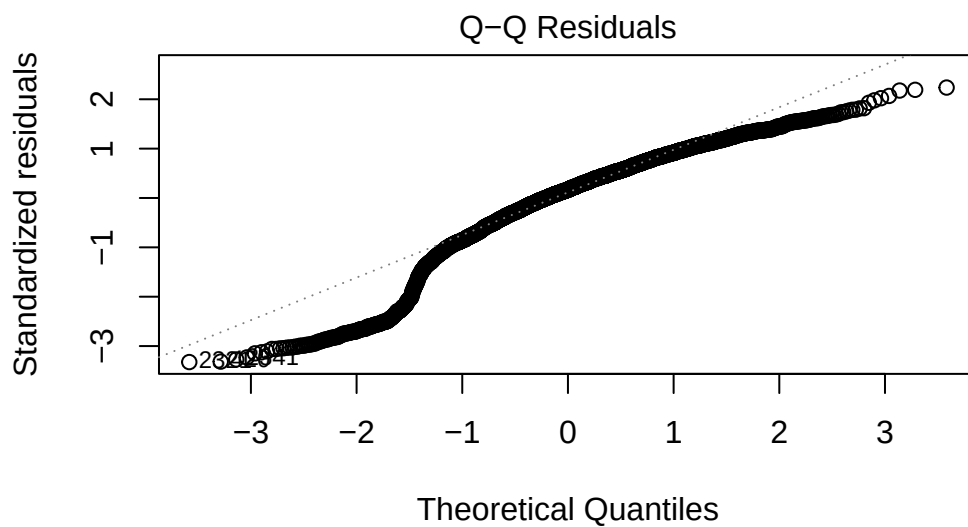
F-statistic: 41.76 on 7 and 2939 DF, p-value: < 2.2e-16

```
plot(track)
```

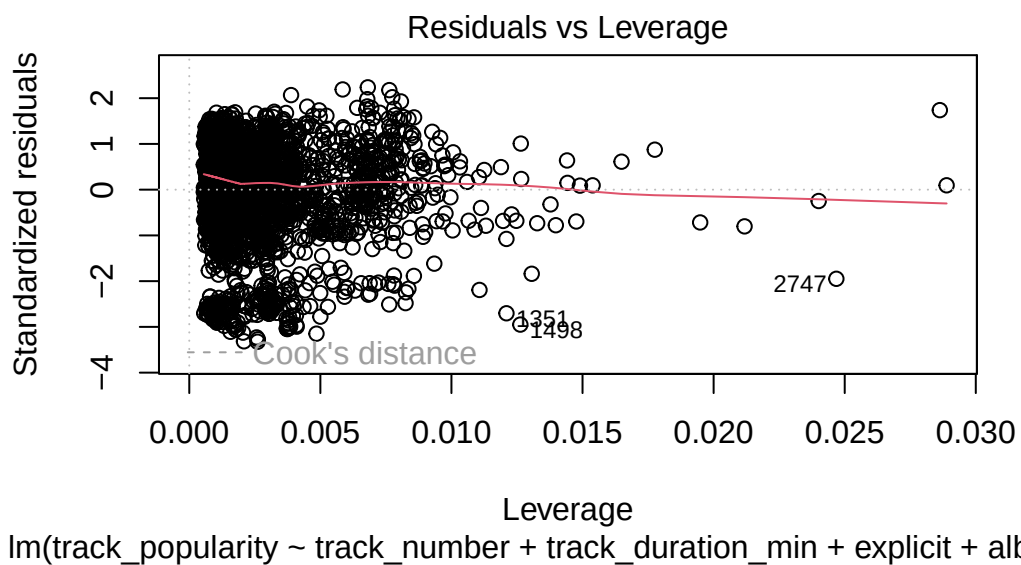
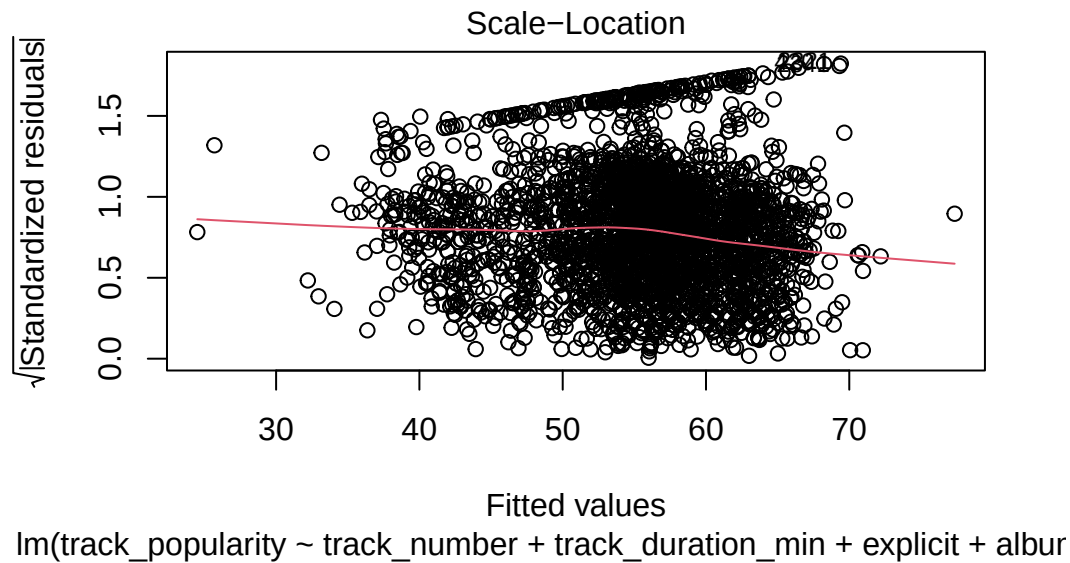




$\text{lm}(\text{track\_popularity} \sim \text{track\_number} + \text{track\_duration\_min} + \text{explicit} + \text{albur})$



$\text{lm}(\text{track\_popularity} \sim \text{track\_number} + \text{track\_duration\_min} + \text{explicit} + \text{albur})$



### Track-only: checking assumptions of NLM

Assumptions violated

```
resid <- residuals(track)
fitted <- fitted(track)

n_bins <- 10
```

```

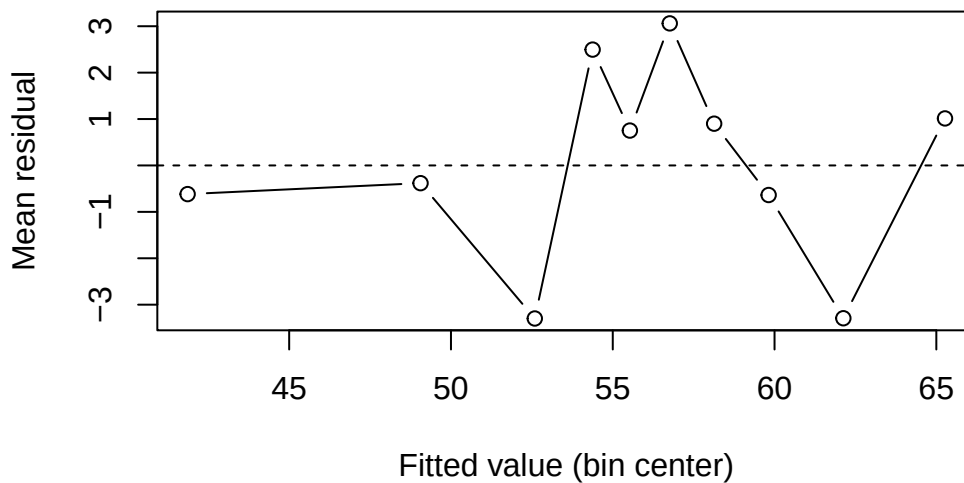
bins <- cut(fitted, breaks = quantile(fitted,
                                      probs = seq(0,1,length.out=n_bins+1)),
           include.lowest = TRUE)

mean_resid <- tapply(resid, bins, mean)

bin_centers <- tapply(fitted, bins, mean)
plot(bin_centers, mean_resid, type = "b",
     xlab = "Fitted value (bin center)",
     ylab = "Mean residual",
     main = "Track-only binned mean residuals vs fitted values")
abline(h = 0, lty = 2)

```

### Track-only binned mean residuals vs fitted values



### Track-only: predictions

```

#assess predictions of track-only model
predictions = predict(track, newdata = test)
track_rmse = sqrt(mean((test$track_popularity - predictions)^2))
track_r2 = cor(test$track_popularity, predictions) ^ 2
SD_y_test = sd(test$track_popularity)
cat('SD of track popularity: ', SD_y_test)

```

SD of track popularity: 21.75762

```
cat('\n track-only RMSE: ', track_rmse)
```

track-only RMSE: 20.42139

```
cat('\n track-only R2: ', track_r2)
```

track-only R2: 0.119248

### Track-only: prediction intervals

```
#number of replications, 1000 replications for each observation in the test set
rep = 1000
boot_outcomes = matrix(NA, nrow = rep, ncol = nrow(test))

#find y = y_hat + residuals 1000 times
for(i in 1:rep){
  #resample the rows from training data with replacement and find new fit
  boot_sample = sample(nrow(train), size = nrow(train), replace = TRUE)
  boot_train = train[boot_sample, ]
  track_boot = lm(track_popularity ~ track_number +
                  track_duration_min + explicit +
                  album_type + log(album_total_tracks) +
                  days_since_release, data = boot_train)
  #find the predictions for the test set using this new fit
  y_hats = predict(track_boot, newdata = test)
  #to find the new outcomes, randomly sample with replacement the boot residuals
  noise = sample(residuals(track_boot), size = nrow(test), replace = TRUE)
  #outcome = fitted_values + noise
  boot_outcomes[i, ] = y_hats + noise
}

lower = apply(boot_outcomes, 2, quantile, probs = 0.025)
upper = apply(boot_outcomes, 2, quantile, probs = 0.975)
width = upper - lower
PI = data.frame(lower = lower, upper = upper, width = width)
head(PI)
```

|   | lower       | upper    | width    |
|---|-------------|----------|----------|
| 1 | 0.7982497   | 84.46972 | 83.67147 |
| 2 | -16.1569980 | 68.85442 | 85.01142 |
| 3 | -1.3901470  | 82.19451 | 83.58466 |
| 4 | -0.2884883  | 84.74430 | 85.03279 |
| 5 | 1.4145078   | 86.13998 | 84.72547 |
| 6 | 8.3039185   | 90.92743 | 82.62351 |

```
track_avg_width = mean(width)
cat('PI average width for the track-only model: ',
    track_avg_width, '\n Track-only PI average bounds bounds:
    [' , mean(lower), ', ', mean(upper), ']')
```

```
PI average width for the track-only model: 84.21112
Track-only PI average bounds bounds:
[ 0.6805937 , 84.89172 ]
```

## Full model

### Full model: fitting model

```
#full model
predictors_formula = paste(c(artist_predictors, 'track_number',
                             'track_duration_min', 'explicit', 'album_type',
                             'log(album_total_tracks)', 'days_since_release'),
                           collapse = ' + ')

full_model_formula = as.formula(paste('track_popularity ~ ', predictors_formula))
full_model = lm(full_model_formula, data = train)
summary(full_model)
```

Call:

```
lm(formula = full_model_formula, data = train)
```

Residuals:

| Min     | 1Q     | Median | 3Q     | Max    |
|---------|--------|--------|--------|--------|
| -68.336 | -7.011 | 3.184  | 12.205 | 42.638 |

Coefficients:

|                         | Estimate   | Std. Error | t value | Pr(> t ) |     |
|-------------------------|------------|------------|---------|----------|-----|
| (Intercept)             | 5.704e+01  | 5.691e+00  | 10.022  | < 2e-16  | *** |
| dark.r.b                | -8.012e+00 | 2.765e+00  | -2.898  | 0.00379  | **  |
| pop.punk                | -4.122e+00 | 3.452e+00  | -1.194  | 0.23243  |     |
| indie                   | 5.411e+00  | 2.699e+00  | 2.005   | 0.04504  | *   |
| edm                     | -3.078e+00 | 2.473e+00  | -1.245  | 0.21327  |     |
| country                 | -1.914e+00 | 1.496e+00  | -1.279  | 0.20090  |     |
| pop                     | -3.769e+00 | 1.582e+00  | -2.382  | 0.01728  | *   |
| folk                    | -1.319e+01 | 3.050e+00  | -4.325  | 1.58e-05 | *** |
| egyptian.pop            | -1.301e+01 | 2.596e+00  | -5.014  | 5.65e-07 | *** |
| soft.pop                | -9.473e+00 | 2.045e+00  | -4.632  | 3.79e-06 | *** |
| pop.rock                | -6.074e+00 | 4.224e+00  | -1.438  | 0.15055  |     |
| alternative.pop         | 7.733e+00  | 2.876e+00  | 2.689   | 0.00721  | **  |
| art.pop                 | -1.022e+01 | 1.895e+00  | -5.394  | 7.43e-08 | *** |
| alternative.r.b         | 1.859e+00  | 4.145e+00  | 0.449   | 0.65380  |     |
| r.b                     | -4.023e+00 | 2.113e+00  | -1.904  | 0.05699  | .   |
| rap                     | -5.175e+00 | 2.074e+00  | -2.495  | 0.01264  | *   |
| soundtrack              | -1.845e+01 | 1.908e+00  | -9.667  | < 2e-16  | *** |
| rock                    | -5.034e+00 | 1.840e+00  | -2.736  | 0.00626  | **  |
| hip.hop                 | -2.895e+00 | 1.524e+00  | -1.900  | 0.05756  | .   |
| nu.metal                | -6.815e+00 | 2.303e+00  | -2.959  | 0.00311  | **  |
| r.b.pop                 | -1.085e+01 | 6.591e+00  | -1.646  | 0.09987  | .   |
| synthpop                | 4.167e+00  | 4.600e+00  | 0.906   | 0.36510  |     |
| indie.pop               | -1.294e+01 | 3.116e+00  | -4.154  | 3.37e-05 | *** |
| grunge                  | -1.457e+01 | 2.907e+00  | -5.011  | 5.74e-07 | *** |
| classic.rock            | -1.687e+00 | 2.762e+00  | -0.611  | 0.54144  |     |
| artist_popularity       | 9.288e-01  | 9.978e-02  | 9.308   | < 2e-16  | *** |
| log_artist_followers    | -3.024e+00 | 6.279e-01  | -4.816  | 1.54e-06 | *** |
| track_number            | -3.342e-01 | 6.309e-02  | -5.297  | 1.27e-07 | *** |
| track_duration_min      | -4.762e-02 | 3.286e-01  | -0.145  | 0.88478  |     |
| explicitTRUE            | 1.592e+00  | 1.012e+00  | 1.573   | 0.11589  |     |
| album_typecompilation   | -4.908e+00 | 1.547e+00  | -3.172  | 0.00153  | **  |
| album_typesingle        | -1.852e+01 | 2.376e+00  | -7.794  | 8.97e-15 | *** |
| log(album_total_tracks) | -4.797e+00 | 8.777e-01  | -5.466  | 5.00e-08 | *** |
| days_since_release      | -6.430e-05 | 1.315e-04  | -0.489  | 0.62485  |     |

---

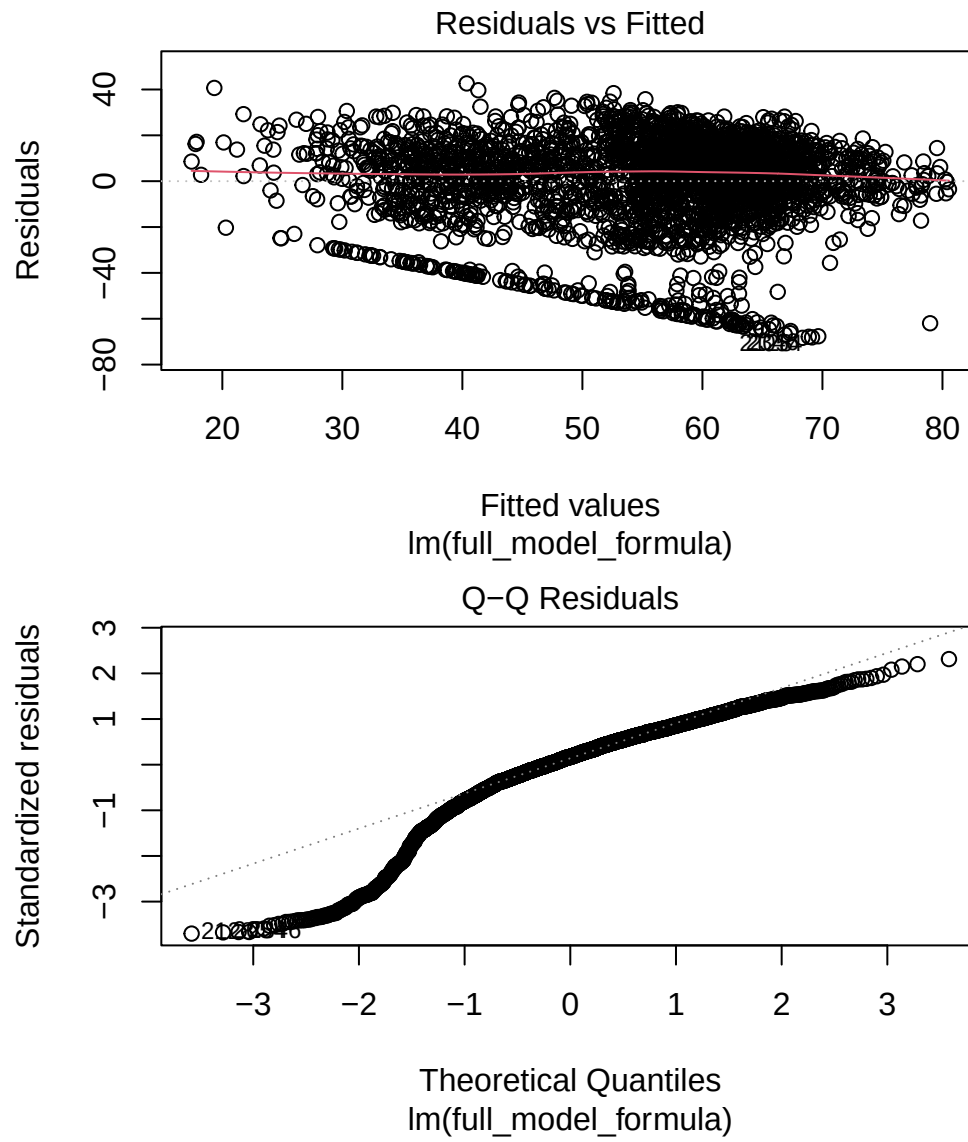
Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

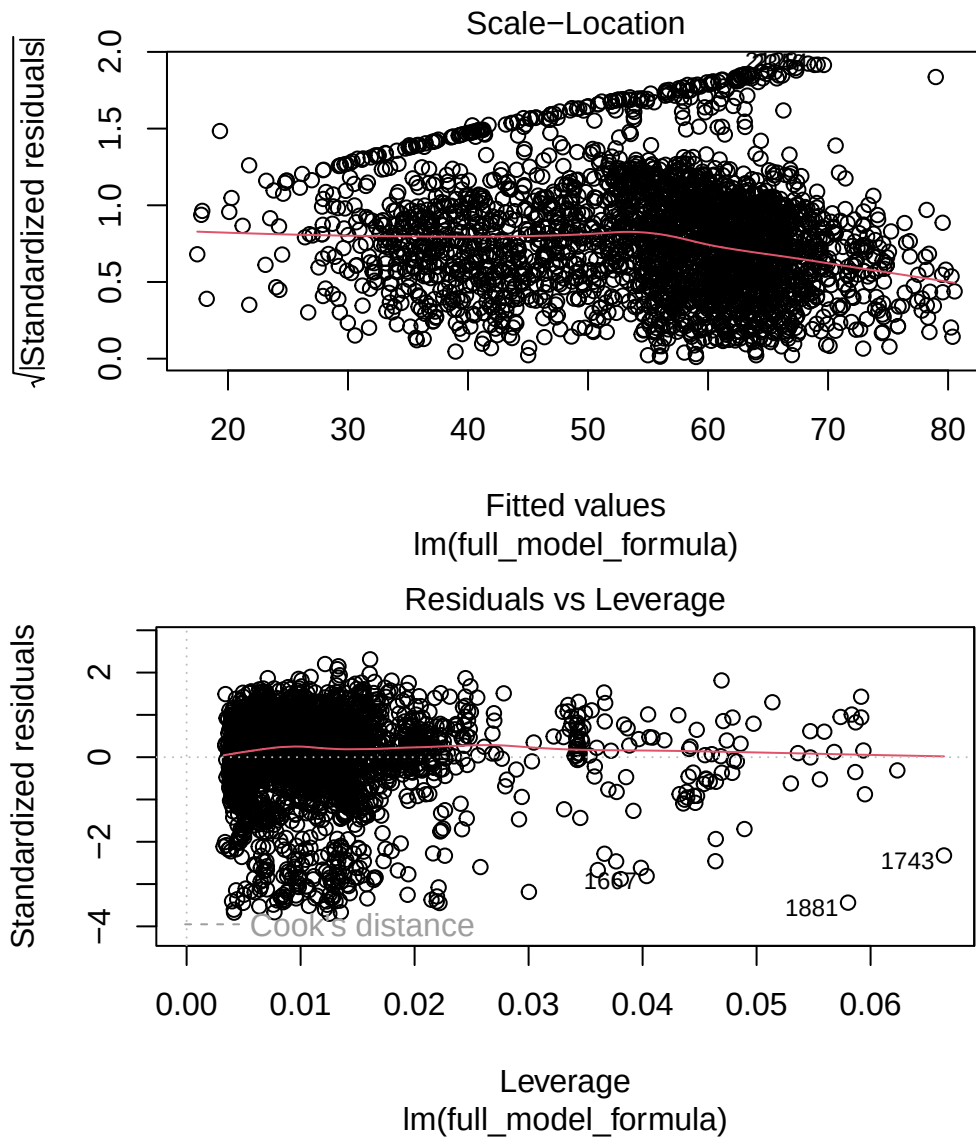
Residual standard error: 18.58 on 2913 degrees of freedom

Multiple R-squared: 0.267, Adjusted R-squared: 0.2587

F-statistic: 32.15 on 33 and 2913 DF, p-value: < 2.2e-16

```
plot(full_model)
```





### Full-model: checking assumptions of NLM

Assumptions violated

```
resid <- residuals(full_model)
fitted <- fitted(full_model)

n_bins <- 10
```



```

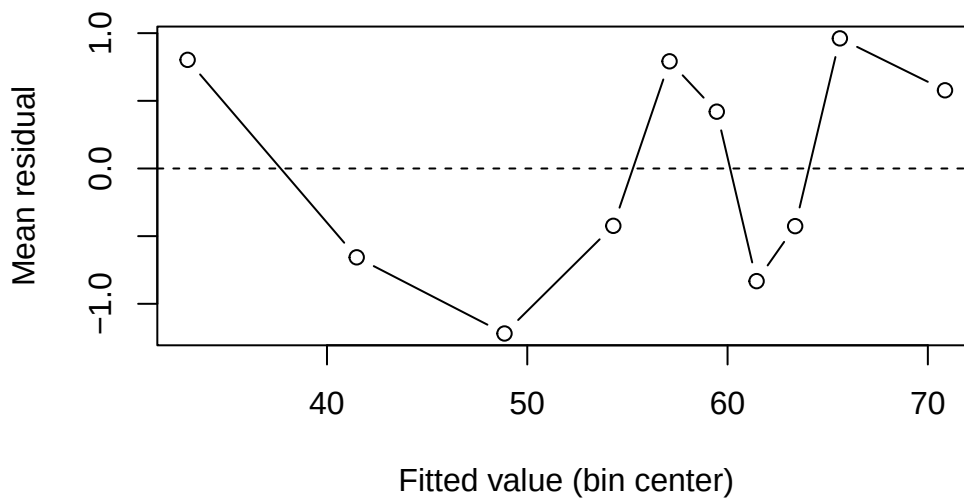
bins <- cut(fitted, breaks = quantile(fitted,
                                      probs = seq(0,1,length.out=n_bins+1)),
           include.lowest = TRUE)

mean_resid <- tapply(resid, bins, mean)

bin_centers <- tapply(fitted, bins, mean)
plot(bin_centers, mean_resid, type = "b",
     xlab = "Fitted value (bin center)",
     ylab = "Mean residual",
     main = "Full model binned mean residuals vs fitted values")
abline(h = 0, lty = 2)

```

**Full model binned mean residuals vs fitted values**



#### full-model: predictions

```

#assess predictions of full model
predictions = predict(full_model, newdata = test)
full_rmse = sqrt(mean((test$track_popularity - predictions)^2))
full_r2 = cor(test$track_popularity, predictions) ^ 2
SD_y_test = sd(test$track_popularity)
cat('SD of track popularity: ', SD_y_test)

```

SD of track popularity: 21.75762

```
cat('\n full model RMSE: ', full_rmse)
```

```
full model RMSE: 18.94407
```

```
cat('\n full model R2: ', full_r2)
```

```
full model R2: 0.2414337
```

### full-model: prediction intervals

```
#number of replications, 1000 replications for each observation in the test set
rep = 1000
boot_outcomes = matrix(NA, nrow = rep, ncol = nrow(test))

#find y = y_hat + residuals 1000 times
for(i in 1:rep){
  #resample the rows from training data with replacement and find new fit
  boot_sample = sample(nrow(train), size = nrow(train), replace = TRUE)
  boot_train = train[boot_sample, ]
  full_boot = lm(full_model_formula, data = boot_train)
  #find the predictions for the test set using this new fit
  y_hats = predict(full_boot, newdata = test)
  #to find the new outcomes, randomly sample with replacement the boot residuals
  noise = sample(residuals(full_boot), size = nrow(test), replace = TRUE)
  #outcome = fitted_values + noise
  boot_outcomes[i, ] = y_hats + noise
}

lower = apply(boot_outcomes, 2, quantile, probs = 0.025)
upper = apply(boot_outcomes, 2, quantile, probs = 0.975)
width = upper - lower
PI = data.frame(lower = lower, upper = upper, width = width)
head(PI)
```

|   | lower       | upper    | width    |
|---|-------------|----------|----------|
| 1 | 7.6118850   | 88.82737 | 81.21548 |
| 2 | -19.6298710 | 62.62375 | 82.25362 |

```

3 -2.3166365 76.74531 79.06195
4 11.8653448 87.33399 75.46865
5 7.0343098 91.07327 84.03896
6 0.7947694 79.41204 78.61727

```

```

full_avg_width = mean(width)
cat('PI average width for the full model: ', full_avg_width,
    '\n Full model PI average bounds bounds: [', mean(lower),
    ', ', mean(upper), ']\n')

```

```

PI average width for the full model: 79.50054
Full model PI average bounds bounds: [ 2.964586 , 82.46513 ]

```

## Interaction model

Meaningful interactions to explore:

**artist\_popularity \* explicit:** If you're already super popular, explicit content doesn't hurt your streams because you have a loyal fan base.

**artist\_popularity \* album\_type:** Popular artists may receive lots of streams, regardless if its a single or album, while less popular artists may perform well on singles but less on albums.

**artist\_popularity \* days\_since\_release:** What if popular artists release music around the same time?

### Interaction model: fitting model

```

predictors_formula = paste(c(artist_predictors, 'track_number',
                             'track_duration_min', 'explicit',
                             'album_type', 'log(album_total_tracks)',
                             'days_since_release',
                             'artist_popularity * explicit',
                             'artist_popularity * album_type',
                             'artist_popularity * days_since_release'),
                           collapse = ' + ')
interaction_model_formula = as.formula(paste('track_popularity ~ ',
                                             predictors_formula))
interaction_model = lm(interaction_model_formula, data = train)
summary(interaction_model)

```

Call:

```
lm(formula = interaction_model_formula, data = train)
```

Residuals:

|  | Min     | 1Q     | Median | 3Q     | Max    |
|--|---------|--------|--------|--------|--------|
|  | -70.957 | -7.140 | 2.881  | 12.103 | 40.546 |

Coefficients:

|                         | Estimate   | Std. Error | t value | Pr(> t ) |
|-------------------------|------------|------------|---------|----------|
| (Intercept)             | 3.743e+01  | 6.509e+00  | 5.750   | 9.83e-09 |
| dark.r.b                | -3.387e+00 | 3.208e+00  | -1.056  | 0.291087 |
| pop.punk                | -2.822e+00 | 3.423e+00  | -0.824  | 0.409762 |
| indie                   | 6.838e+00  | 2.678e+00  | 2.554   | 0.010715 |
| edm                     | -1.572e+00 | 2.463e+00  | -0.638  | 0.523295 |
| country                 | -2.001e+00 | 1.493e+00  | -1.340  | 0.180229 |
| pop                     | -3.627e+00 | 1.574e+00  | -2.304  | 0.021283 |
| folk                    | -1.645e+01 | 3.063e+00  | -5.372  | 8.39e-08 |
| egyptian.pop            | -9.946e+00 | 2.616e+00  | -3.803  | 0.000146 |
| soft.pop                | -8.202e+00 | 2.033e+00  | -4.034  | 5.62e-05 |
| pop.rock                | -5.893e+00 | 4.178e+00  | -1.410  | 0.158508 |
| alternative.pop         | 7.312e+00  | 2.848e+00  | 2.567   | 0.010303 |
| art.pop                 | -9.878e+00 | 1.875e+00  | -5.269  | 1.47e-07 |
| alternative.r.b         | 4.316e+00  | 4.147e+00  | 1.041   | 0.298105 |
| r.b                     | -2.380e+00 | 2.103e+00  | -1.132  | 0.257830 |
| rap                     | -4.266e+00 | 2.098e+00  | -2.033  | 0.042137 |
| soundtrack              | -1.899e+01 | 1.915e+00  | -9.916  | < 2e-16  |
| rock                    | -3.759e+00 | 1.843e+00  | -2.039  | 0.041536 |
| hip.hop                 | -1.412e+00 | 1.527e+00  | -0.924  | 0.355414 |
| nu.metal                | -4.412e+00 | 2.302e+00  | -1.917  | 0.055385 |
| r.b.pop                 | -1.456e+01 | 6.552e+00  | -2.223  | 0.026299 |
| synthpop                | 3.812e+00  | 4.551e+00  | 0.838   | 0.402375 |
| indie.pop               | -1.115e+01 | 3.110e+00  | -3.585  | 0.000342 |
| grunge                  | -1.160e+01 | 2.902e+00  | -3.997  | 6.58e-05 |
| classic.rock            | 2.825e+00  | 2.816e+00  | 1.003   | 0.315949 |
| artist_popularity       | 1.234e+00  | 1.109e-01  | 11.120  | < 2e-16  |
| log_artist_followers    | -3.434e+00 | 6.234e-01  | -5.508  | 3.95e-08 |
| track_number            | -3.539e-01 | 6.252e-02  | -5.661  | 1.65e-08 |
| track_duration_min      | 7.111e-02  | 3.259e-01  | 0.218   | 0.827301 |
| explicitTRUE            | -1.665e+01 | 6.293e+00  | -2.646  | 0.008194 |
| album_typecompilation   | 6.634e+00  | 8.155e+00  | 0.813   | 0.416043 |
| album_typesingle        | -9.747e+00 | 7.124e+00  | -1.368  | 0.171343 |
| log(album_total_tracks) | -4.042e+00 | 8.776e-01  | -4.606  | 4.28e-06 |

|                                         |            |           |        |          |
|-----------------------------------------|------------|-----------|--------|----------|
| days_since_release                      | 4.960e-03  | 6.945e-04 | 7.141  | 1.16e-12 |
| artist_popularity:explicitTRUE          | 2.086e-01  | 7.450e-02 | 2.801  | 0.005135 |
| artist_popularity:album_typecompilation | -1.700e-01 | 1.129e-01 | -1.506 | 0.132132 |
| artist_popularity:album_typesingle      | -9.257e-02 | 8.618e-02 | -1.074 | 0.282808 |
| artist_popularity:days_since_release    | -6.821e-05 | 9.330e-06 | -7.311 | 3.42e-13 |
| (Intercept)                             | ***        |           |        |          |
| dark.r.b                                |            |           |        |          |
| pop.punk                                |            |           |        |          |
| indie                                   | *          |           |        |          |
| edm                                     |            |           |        |          |
| country                                 |            |           |        |          |
| pop                                     | *          |           |        |          |
| folk                                    | ***        |           |        |          |
| egyptian.pop                            | ***        |           |        |          |
| soft.pop                                | ***        |           |        |          |
| pop.rock                                |            |           |        |          |
| alternative.pop                         | *          |           |        |          |
| art.pop                                 | ***        |           |        |          |
| alternative.r.b                         |            |           |        |          |
| r.b                                     |            |           |        |          |
| rap                                     | *          |           |        |          |
| soundtrack                              | ***        |           |        |          |
| rock                                    | *          |           |        |          |
| hip.hop                                 |            |           |        |          |
| nu.metal                                | .          |           |        |          |
| r.b.pop                                 | *          |           |        |          |
| synthpop                                |            |           |        |          |
| indie.pop                               | ***        |           |        |          |
| grunge                                  | ***        |           |        |          |
| classic.rock                            |            |           |        |          |
| artist_popularity                       | ***        |           |        |          |
| log_artist_followers                    | ***        |           |        |          |
| track_number                            | ***        |           |        |          |
| track_duration_min                      |            |           |        |          |
| explicitTRUE                            | **         |           |        |          |
| album_typecompilation                   |            |           |        |          |
| album_typesingle                        |            |           |        |          |
| log(album_total_tracks)                 | ***        |           |        |          |
| days_since_release                      | ***        |           |        |          |
| artist_popularity:explicitTRUE          | **         |           |        |          |
| artist_popularity:album_typecompilation |            |           |        |          |
| artist_popularity:album_typesingle      |            |           |        |          |

```
artist_popularity:days_since_release ***
```

```
---
```

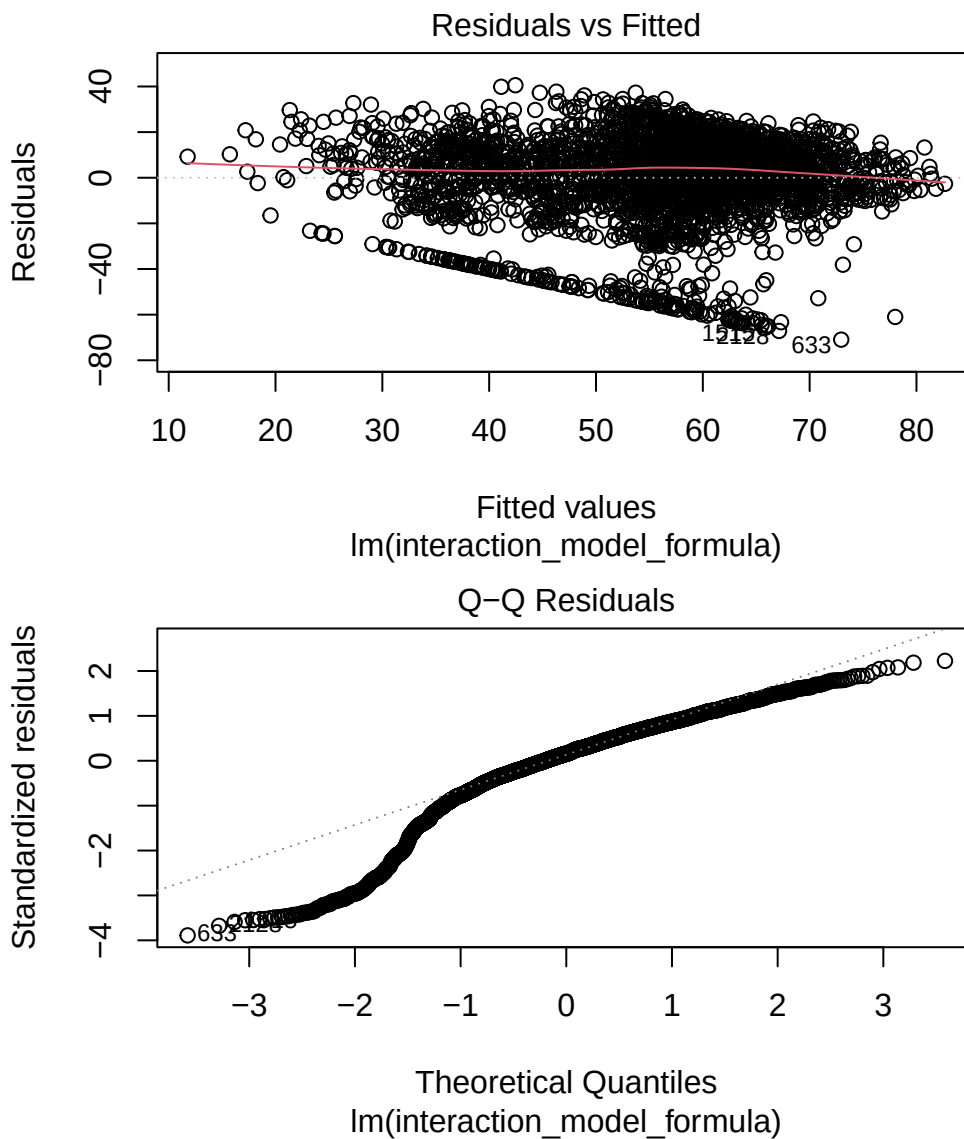
```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

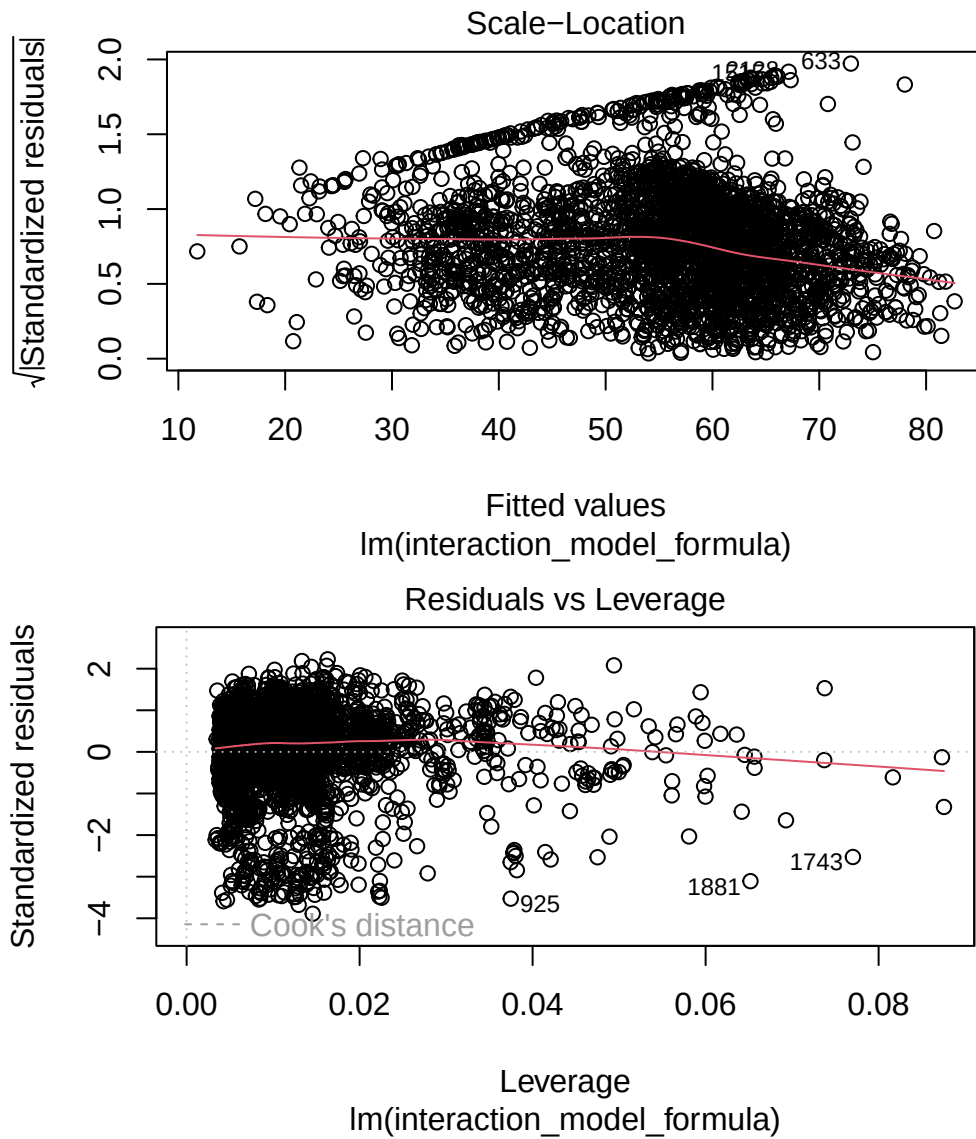
```
Residual standard error: 18.37 on 2909 degrees of freedom
```

```
Multiple R-squared:  0.2842,    Adjusted R-squared:  0.2751
```

```
F-statistic: 31.21 on 37 and 2909 DF,  p-value: < 2.2e-16
```

```
plot(interaction_model)
```





### Interaction model: checking assumptions of NLM

Assumptions violated

```
resid <- residuals(interaction_model)
fitted <- fitted(interaction_model)

n_bins <- 10
```

```

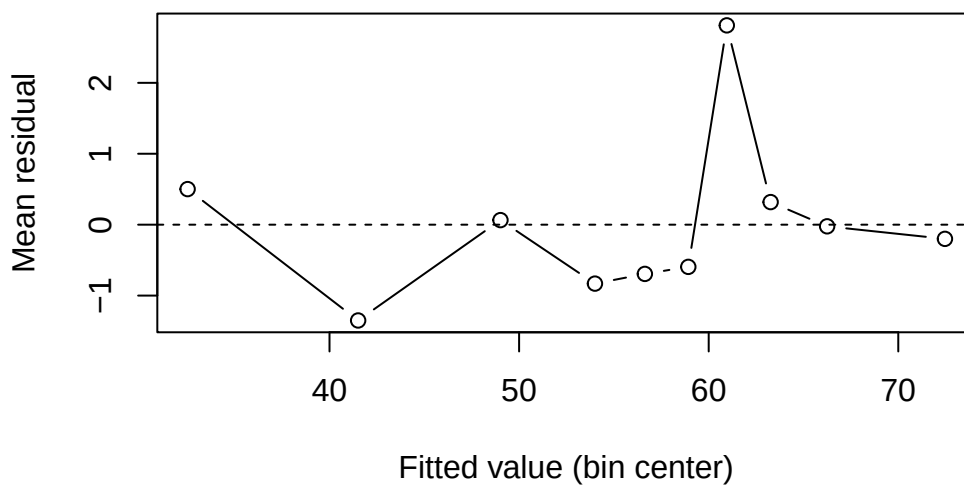
bins <- cut(fitted, breaks = quantile(fitted, probs =
                                     seq(0,1,length.out=n_bins+1)),
           include.lowest = TRUE)

mean_resid <- tapply(resid, bins, mean)

bin_centers <- tapply(fitted, bins, mean)
plot(bin_centers, mean_resid, type = "b",
     xlab = "Fitted value (bin center)",
     ylab = "Mean residual",
     main = "Interaction model binned mean residuals vs fitted values")
abline(h = 0, lty = 2)

```

### Interaction model binned mean residuals vs fitted value:



### Interaction model: predictions

```

#assess predictions of full model
predictions = predict(interaction_model, newdata = test)
interaction_rmse = sqrt(mean((test$track_popularity - predictions)^2))
interaction_r2 = cor(test$track_popularity, predictions) ^ 2
SD_y_test = sd(test$track_popularity)
cat('SD of track popularity: ', SD_y_test)

```

SD of track popularity: 21.75762



```
cat('\n interaction model RMSE: ', interaction_rmse)
```

interaction model RMSE: 18.75385

```
cat('\n interaction model R2: ', interaction_r2)
```

interaction model R2: 0.2567235

### Interaction model: prediction intervals

```
#number of replications, 1000 replications for each observation in the test set
rep = 1000
boot_outcomes = matrix(NA, nrow = rep, ncol = nrow(test))

#find y = y_hat + residuals 1000 times
for(i in 1:rep){
  #resample the rows from training data with replacement and find new fit
  boot_sample = sample(nrow(train), size = nrow(train), replace = TRUE)
  boot_train = train[boot_sample, ]
  interaction_boot = lm(interaction_model_formula, data = boot_train)
  #find the predictions for the test set using this new fit
  y_hats = predict(interaction_boot, newdata = test)
  #to find the new outcomes, randomly sample with replacement the boot residuals
  noise = sample(residuals(interaction_boot), size = nrow(test), replace = TRUE)
  #outcome = fitted_values + noise
  boot_outcomes[i, ] = y_hats + noise
}

lower = apply(boot_outcomes, 2, quantile, probs = 0.025)
upper = apply(boot_outcomes, 2, quantile, probs = 0.975)
width = upper - lower
PI = data.frame(lower = lower, upper = upper, width = width)
head(PI)
```

|   | lower      | upper    | width    |
|---|------------|----------|----------|
| 1 | 6.097997   | 86.74221 | 80.64421 |
| 2 | -22.324864 | 58.90514 | 81.23000 |

```

3 -6.596637 75.43354 82.03018
4 5.831183 85.78875 79.95757
5 10.135294 87.89885 77.76355
6 -2.038656 79.28032 81.31897

```

```

interaction_avg_width = mean(width)
cat('PI average width for the interaction model: ',
    interaction_avg_width, '\n Interaction model PI average bounds bounds:
    [' , mean(lower), ', ', mean(upper), ' ]')

```

```

PI average width for the interaction model: 78.87647
Interaction model PI average bounds bounds:
[ 3.594648 , 82.47112 ]

```

## Comparing Models

```

models = c('artist-only', 'track-only', 'full', 'interaction')
model_RMSE = c(artist_rmse, track_rmse, full_rmse, interaction_rmse)
model_r2 = c(artist_r2, track_r2, full_r2, interaction_r2)
model_avg_width = c(artist_avg_width, track_avg_width, full_avg_width,
                    interaction_avg_width)
models_df = data.frame(models = models, RMSE = model_RMSE, R2 = model_r2,
                      avg_PI_width = model_avg_width)
models_df %>% arrange(RMSE)

```

|   | models      | RMSE     | R2        | avg_PI_width |
|---|-------------|----------|-----------|--------------|
| 1 | interaction | 18.75385 | 0.2567235 | 78.87647     |
| 2 | full        | 18.94407 | 0.2414337 | 79.50054     |
| 3 | artist-only | 19.51403 | 0.1960646 | 79.51638     |
| 4 | track-only  | 20.42139 | 0.1192480 | 84.21112     |

Low RMSE indicates better accuracy.  $R^2$  is what proportion of the variability in track popularity is explained by the model. A narrow prediction interval has greater precision in predicted track popularity scores.

The interaction model has the lowest RMSE, highest  $R^2$ , and the most narrow prediction interval, therefore it's the model that performs best at predicting track popularity.

## Testing our interaction model across artists for a fixed track

```
set.seed(151)
#for a fixed track that's representative, we use the median of numeric features
#and the reference group of categorical/binary variables
fixed_track = data.frame(track_number = median(train$track_number),
                          track_duration_min = median(train$track_duration_min),
                          explicit = factor(FALSE), album_type = factor('album'),
                          album_total_tracks = median(train$album_total_tracks),
                          days_since_release = median(train$days_since_release))

#selecting the least popular artist in training set, the mid point,
#and the most popular artist
artist_profiles = train %>% select(artist_name, artist_popularity,
                                   log_artist_followers, top_genre_names_new) %>%
  distinct() %>% arrange(artist_popularity)
three_artist_profiles = artist_profiles %>%
  slice(c(1, round(nrow(artist_profiles)/2), nrow(artist_profiles)))

#bind columns together of fixed track and artist profiles
artists_and_track = three_artist_profiles %>% bind_cols(fixed_track)

#predict track popularity
artists_and_track$predicted_track_pop = predict(interaction_model,
                                                  newdata = artists_and_track)
artists_and_track %>% select(artist_name, predicted_track_pop,
                             artist_popularity, log_artist_followers)
```

```
# A tibble: 3 x 4
  artist_name predicted_track_pop artist_popularity log_artist_followers
  <chr>          <dbl>          <dbl>          <dbl>
1 Lafawndah      13.7             16             9.97
2 Tim McGraw     55.9             69            15.4
3 Taylor Swift   62.2            100            18.8
```

Based on the predictions of our model, the artist with the greatest track popularity for the fixed song is Taylor Swift.